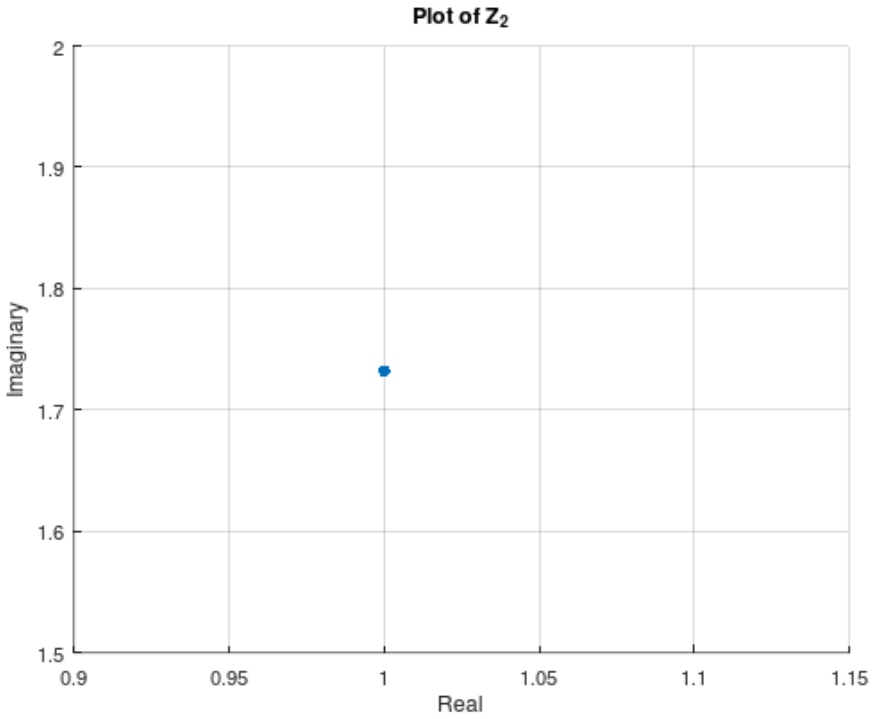
Lab 1 results document:

Name: Anas Khmais Hamrouni
Names of all students you discussed this lab with:

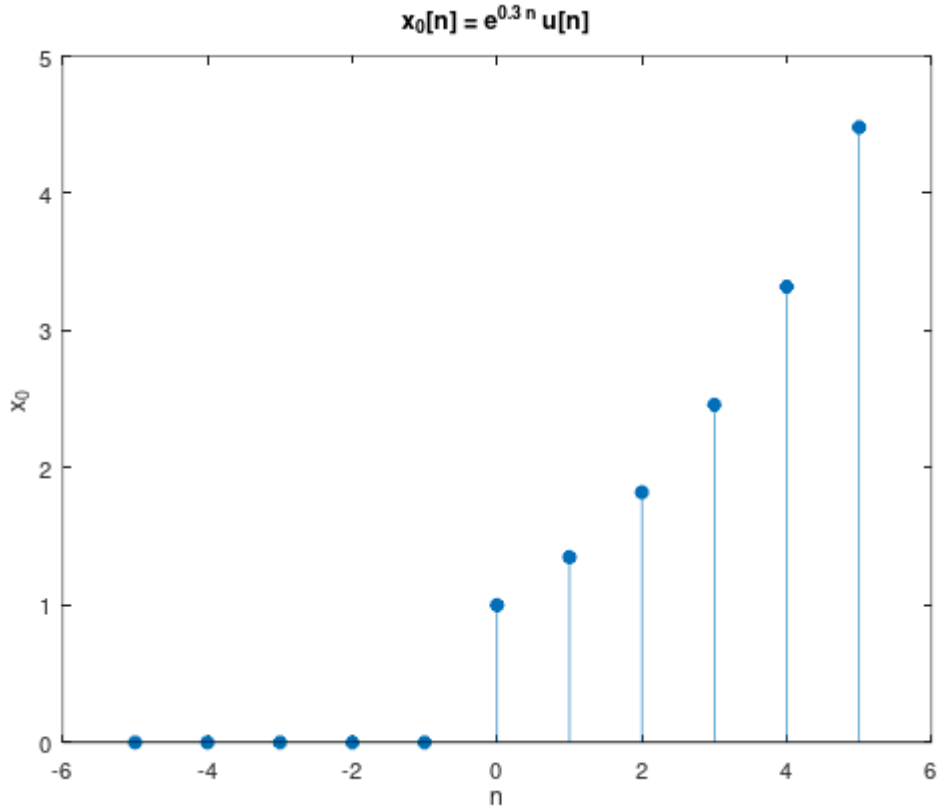
Section 1:
Output from item 1:
```
ans =  0 + 1i
```

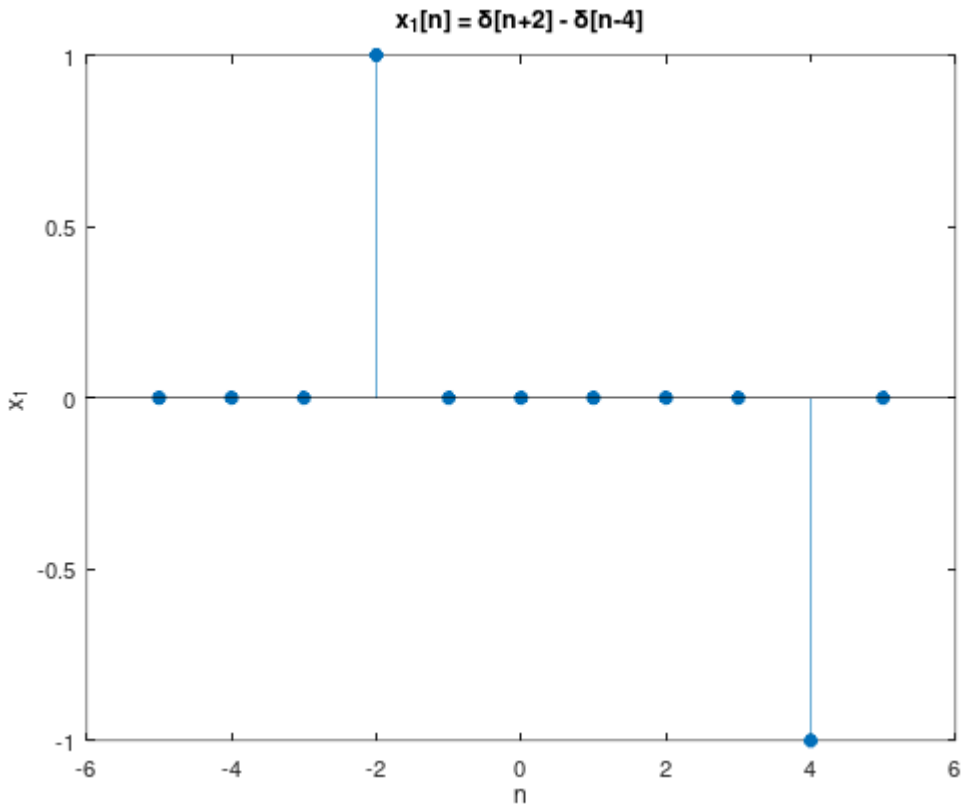
Output from item 2:
```
ans =  0 + 2i
ans =  0 + 2i
```
Output from item 3:
```
>> z1=1+1j
z1 =  1 + 1i
>> abs(z1)
ans = 1.4142
>> angle(z1)
ans = 0.7854
>> real(z1)
ans = 1
>> imag(z1)
ans = 1
```
Answer to item 4:
-> The `angle()` function returns the phase in radians
Output from item 5:
```
>> abs(z1+z2)
ans = 3.3859
```
Plot from item 6:

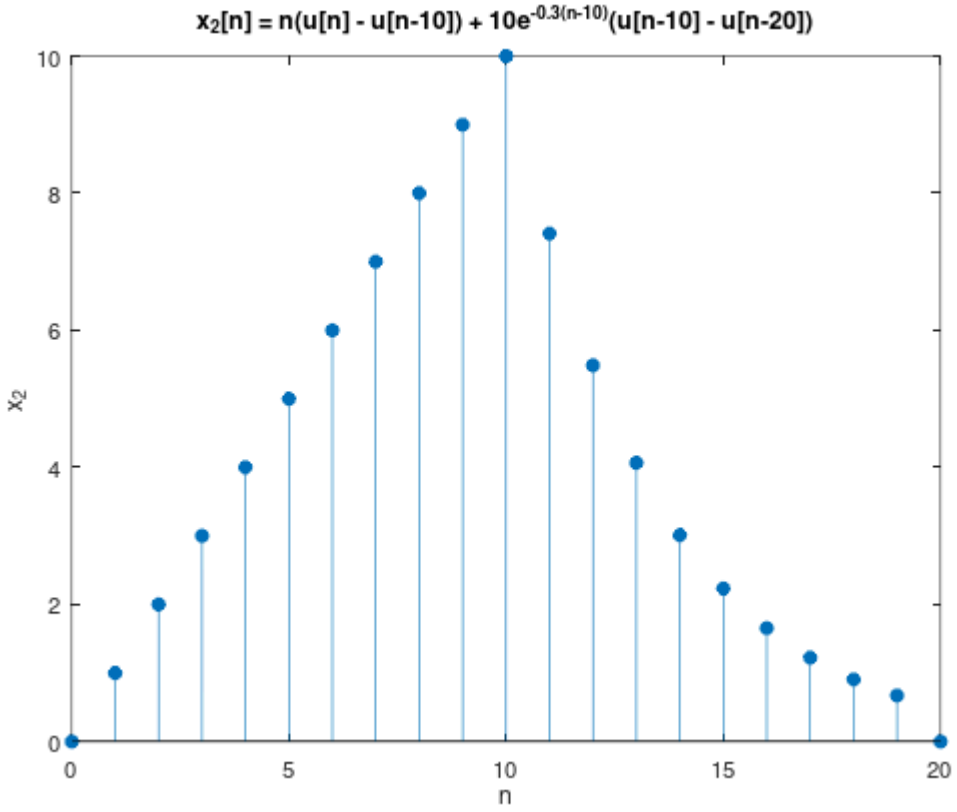


Section 2:

Include your figures for parts (a), (b), and (c), below.

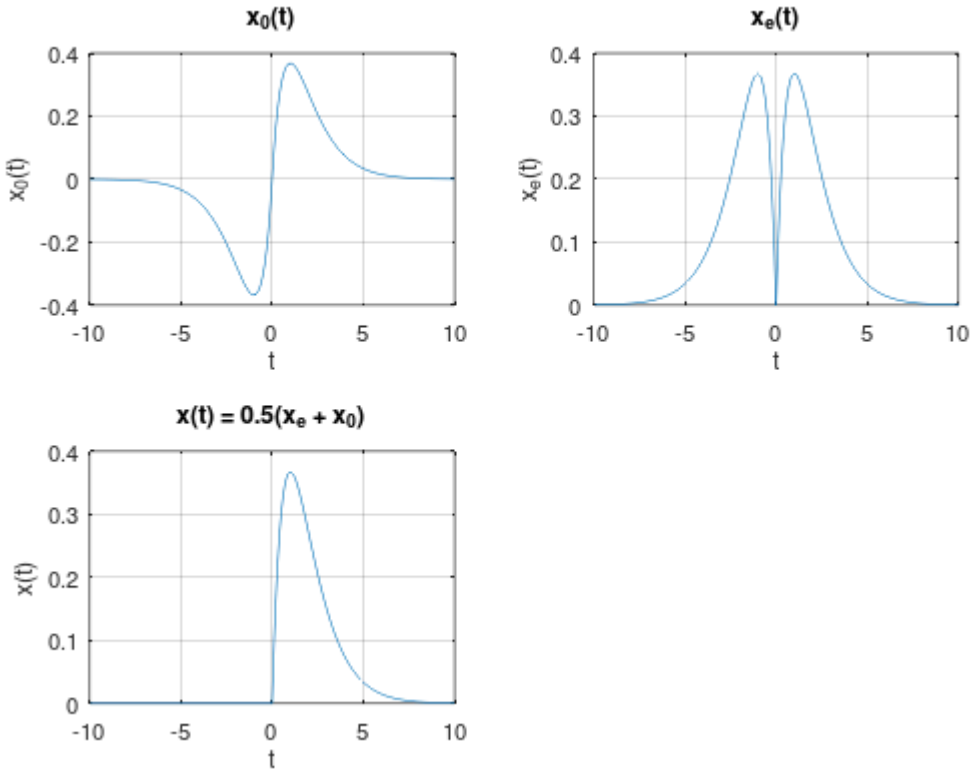



x1[n] = δ[n+2] - δ[n-4]
1
0.5
0
-0.5
-1
x1
-6
-4
-2
0
2
4
6
n


x2[n] = n(u[n] - u[n-10]) + 10e-0.3(n-10)(u[n-10] - u[n-20])
10
8
6
4
2
0
x2
0
5
10
15
20
n

Section 3:

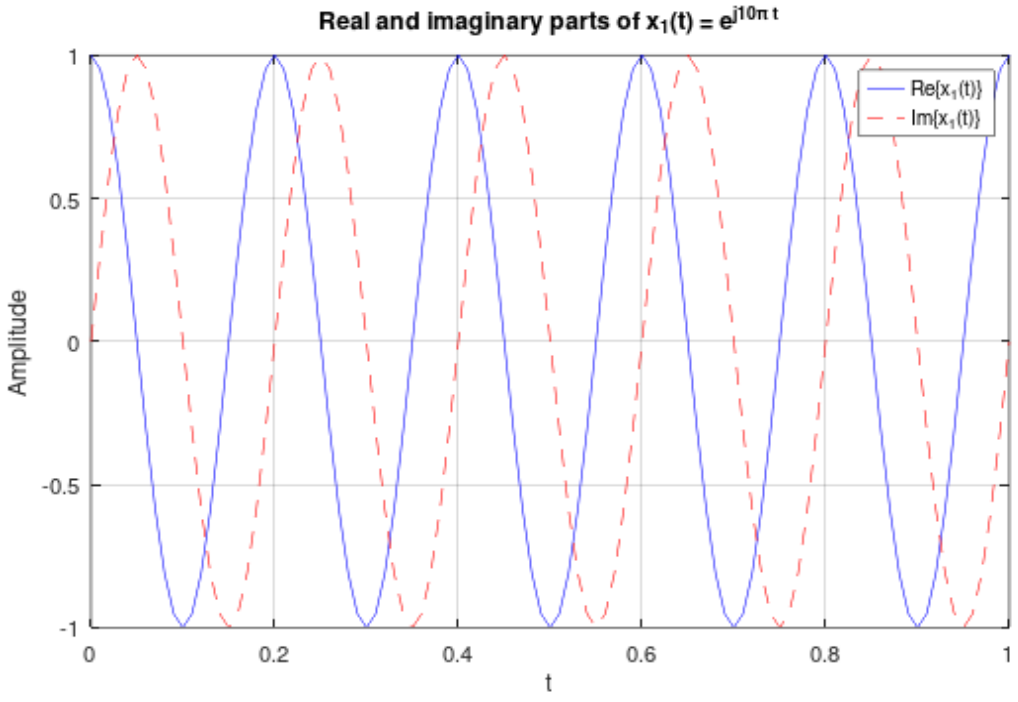
Include your figure from item 3, below:



Answer to item 4:

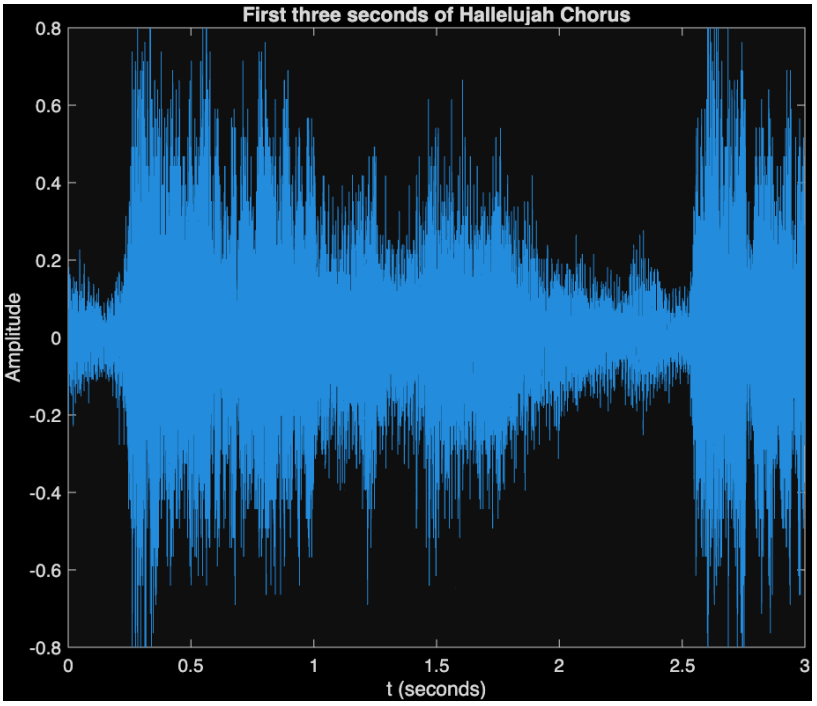
-> The signal x_e is even (symmetric around t=0) while signal x_0 *is odd*

Include your figure from item 6, below:



Section 4

Include your figure from item 6, below:



Question 2: Is the sound higher or lower than before? Why do you think this is happening?

-> The sound pitch seemed lower. This is because we used the same samples but played them at a lower sampling frequency, so the same waveform was stretched over more time. As a result, there were fewer signal cycles per second (lower effective frequency), and the pitch sounded lower.

Question 3a: Use `sound()` *to play the signal ys = 2xs. What do you hear? Why do you think you hear this?*

-> I hear distortion. This is because the amplitude of the signal ys = 2xs goes outside the range [-1, 1], so the samples are clipped during playback, which causes audible distortion.

Question 3b: Use `soundsc()` *to play the signal ys = 2xs. What do you hear? Use* `help soundsc` *to explain this.*

-> The sound is now clear and has the same pitch as the original. This is because `soundsc` rescales the signal so that its values lie within [-1, 1] before playback, avoiding clipping.

Question 4: How many samples correspond to the first three seconds of sound of the `y` *signal (including the sound at time t = 0 and time t = 3)?*

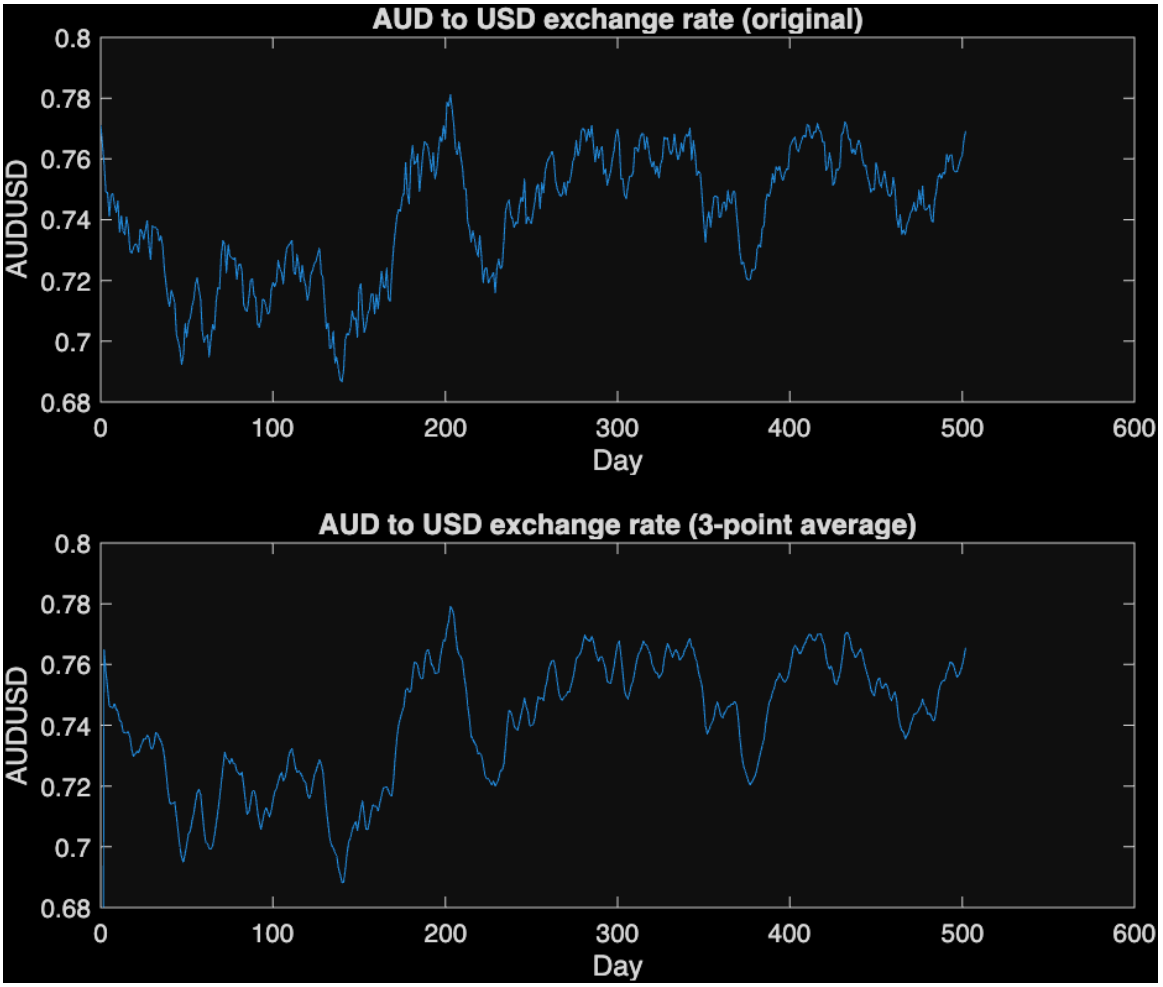
-> N = number of samples = Fs * 3 + 1 = 8192 * 3 + 1 = 24577

Section 5:

Question 4:
-> For N = 4, the required input time m = n − 4 is always outside 0..3 for all n in 0..3, so all outputs become 0

Question: Does your `sumsys()` *system also work for continuous-time signals?*
-> `SumSys()` adds samples of signals, so if the time vectors of these samples are the same it works.

Include your plot of `aud` *and* `audout` *from item 6(b), below:*



-> The new signal is a bit smoother with less noise

Code for section 1:

Code from section 1.1:

```
Paste your code in here.
>> sqrt(-1)

>> i + j
>> 1i + 1j

>> z1=1+1j
>> abs(z1)
>> angle(z1)
>> real(z1)
>> imag(z1)

>> z2 = 2 * exp( j * pi / 3 )
>> abs(z1+z2)

>> scatter ( real(z2), imag(z2), 'filled' )
>> grid on
>> xlabel('Real')
>> ylabel('Imaginary')
>> title('Plot of Z_2')
```

Code for section 2

Code for function `dtstep`:

```
Paste your code here.

function [x, n] = dtstep (n0, n1, n2)
% dtstep: returns discrete-time unit step function
% [x, n] = dtstep(n0,n1,n2)
% produces x[n] = u[n - n0] for n1 <= n <= n2
n = n1:n2;
x = zeros(1,length(n));
x(n>=n0) = 1;
endfunction
```

Code for section 2 item 2 (script to plot three discrete-time signals):

```
Paste your code here.

n0 = 0; n1= -5; n2= 5;
[u, n] = dtstep(n0, n1, n2)
x0 = exp(0.3 * n) .* u
figure;
stem(n , x0, 'filled');
xlabel('n');
ylabel('x_0');
title('x_0[n] = e^{0.3 n} u[n]');

[x11, n] = dtimpulse(-2, n1, n2)
[x12, n] = dtimpulse(4, n1, n2)
x1 = x11 - x12;
figure;
stem(n , x1, 'filled');
xlabel('n');
ylabel('x_1');
title('x_1[n] = \delta[n+2] - \delta[n-4]');

n1 = 0 ; n2 = 20;
[u1, n] = dtstep(n0, n1, n2)
[u2, n] = dtstep(10, n1, n2)
[u3, n] = dtstep(20, n1, n2)
x2 = n .* ( u1 - u2 ) + ( 10 * exp( -0.3 * (n-10) ) .* ( u2 - u3 )
)
figure;
stem(n , x2, 'filled');
xlabel('n');
ylabel('x_2');
title('x_2[n] = n(u[n] - u[n-10]) + 10e^{-0.3(n-10)}(u[n-10] -
u[n-20])');
```

Code for section 3

Code from section 3.2:

```
Paste your code in here.

t = linspace( -10, 10, 201);

x0 = t .* exp( -abs(t) );

figure;
subplot(2,2, 1);
plot(t, x0);
grid on;
title('x_0(t)');
xlabel('t'); ylabel('x_0(t)');

xe = abs(t) .* exp( -abs(t) );
subplot(2,2,2);
plot(t, xe);
grid on;
title('x_e(t)');
xlabel('t'); ylabel('x_e(t)');

x = 0.5 .* (xe + x0);
subplot(2,2,3);
plot(t, x);
grid on;
title('x(t) = 0.5(x_e + x_0)');
xlabel('t'); ylabel('x(t)');

t = linspace(0, 1, 101);

x1 = exp(j * 10 * pi * t);

x1_real = real(x1);
x1_imag = imag(x1);

figure;
plot(t, x1_real, 'b');
hold on;
plot(t, x1_imag, 'r--');
hold off;

grid on;
xlabel('t');
ylabel('Amplitude');
title('Real and imaginary parts of x_1(t) = e^{j10\pi t}');
legend('Re\{x_1(t)\}', 'Im\{x_1(t)\}');
```

Code for section 4:

Code from section 4.2:

```
Paste your code in here.

fs = 8192;
ts = 3*fs + 1;
t = linspace(0, 3, ts);
xs = cos(2*pi*440*t);
ys = 2 .* xs;
%soundsc(ys, fs);
load handel
N = Fs*3 + 1;
y3 = y(1:N);
sound(y3, Fs);
ty3 = linspace(0, 3, length(y3));
plot(ty3, y3);
title('First three seconds of Hallelujah Chorus');
xlabel('t (seconds)');
ylabel('Amplitude');
```

Code for section 5:

Code for the function `sumsys()`

```
function [y, nout] = sumsys(x1, nin1, x2, nin2)
% sumsys: implements sum system
% [y, nout] = sumsys(x1, nin1, x2, nin2)
% where x1, x2 are row vectors and nin1, nin2 are their time
indices
% (same length and identical entries), produces y[n] = x1[n] +
x2[n]
% for all n, and returns the common time index vector nout.
% Check lengths match
if length(x1) ~= length(nin1) || length(x2) ~= length(nin2)
   error('Signal and time vectors must have the same length for
each input');
end
if any(nin1 ~= nin2)
   error('Time index vectors nin1 and nin2 must be identical');
end
nout = nin1;
y = x1 + x2;
```

Code for the function `delaysys()`

```
function [y, ny] = delaysys(N, x, nx)
% delaysys: implements delay system y[n] = x[n-N]
% [y, ny] = delaysys(N, x, nx)
% N  : non-negative integer delay
% x  : row vector of samples
% nx : row vector of time indices, same length as x
% y, ny : delayed output signal and its time indices (ny = nx).
%
% Assumes x[n] = 0 for times outside nx

if length(x) ~= length(nx)
   error('arguments 2 and 3 should have the same length');
end
if ~isscalar(N) || N < 0 || floor(N) ~= N
   error('argument 1 should be a non-negative integer');
end
ny = nx;
y  = zeros(size(x));
for k = 1:length(nx)
   n = nx(k);
   m = n - N;
   idx = find(nx == m, 1);
   if ~isempty(idx)
       y(k) = x(idx);
   else
```

```
        y(k) = 0;          % outside range
    end
End
```

Code for the function `threepointaverage()`

```
function [y, ny] = threepointaverage(x, nx)
% [y, ny] = threepointaverage(x, nx)
% x, nx : input signal and its time indices (row vectors, same
length)
% y, ny : output signal and its time indices (same as nx)
if length(x) ~= length(nx)
   error('x and nx must have the same length');
end
[x1, n1] = gainsys(1/3, x, nx);
[x2d, n2d] = delaysys(1, x, nx);
[x2, n2]   = gainsys(1/3, x2d, n2d);
[x3d, n3d] = delaysys(2, x, nx);
[x3, n3]   = gainsys(1/3, x3d, n3d);
[u, nu] = sumsys(x1, n1, x2, n2);
[y, ny] = sumsys(u, nu, x3, n3);
```